DB에서 일기 원문을 숨기기까지

AES-GCM과 HMAC으로 원문 조회와 생성 재시도를 유지한 설계

아이큐(한동희) | 테크니컬 라이팅 최종본 | 2026-11-09

Java로 서버를 개발하며 저장 데이터에 암호화를 도입하려는 개발자를 위한 글이다. 원문을 숨기는 방법뿐 아니라, 다시 읽고 같은 요청을 재시도하는 기능까지 함께 다룬다.

1. 원문을 숨겨도 원문에서 만든 지문은 남는다

하루들은 사용자의 일기를 네 컷 만화로 만드는 서비스다. 서버는 일기 원문을 DB에 저장하고, 본인 상세 조회와 AI 생성에 다시 사용한다. 원문을 그대로 보관하면 DB 조회 권한만으로 사적인 내용을 읽을 수 있다.

이 문제는 원문 열을 암호화하는 것으로 끝나지 않았다. 하루들은 생성 요청이 반복됐는지 확인하려고 원문에서 계산한 **요청 지문**도 저장하고 있었다. 비밀 키 없이 계산하는 지문이라면, 공격자가 예상한 문장으로 같은 값을 만들어 대조할 수 있다. 원문을 직접 읽는 경로와 원문 후보가 맞는지 확인하는 경로를 함께 살펴봐야 했다.

이 글은 원문에는 AES-GCM 암호화를 적용하고, 요청 지문에는 비밀 키가 필요한 HMAC을 적용한 과정을 설명한다. 먼저 무엇을 보호할지 정하고, 저장과 조회, 재시도와 키 교체 순서로 구현을 따라간다. 두 방식의 용어와 동작은 각 절에서 설명한다.

DB와 키를 함께 얻지 못한 경우를 보호한다

이번 목표는 **애플리케이션의 비밀 키 없이 DB 조회 결과나 논리 덤프만 얻은 사람이 신규 일기 원문을 직접 읽거나 짧은 후보를 지문으로 대조하지 못하게 하는 것**이다. 논리 덤프는 SQL 조회 등으로 행과 값을 파일로 내보낸 결과다.

서버는 여전히 원문을 복원할 수 있어야 한다. 작성자에게 일기를 보여 주고 AI에 입력하려면 원문이 필요하기 때문이다. 비밀번호처럼 복원할 수 없는 값만 저장하는 방법은 이 요구에 맞지 않는다. 서버도 원문을 볼 수 없게 하는 End-to-End Encryption(종단간 암호화)을 적용하려면 AI 호출과 기기 간 키 전달, 키 복구부터 다시 설계해야 한다.

```
쓰기: 원문 입력 -> 애플리케이션에서 암호화 -> DB에 암호문 저장
읽기: DB에서 조회 -> 소유권 확인 -> 복호화 -> 작성자에게 응답
키:   DB에 저장하지 않고 애플리케이션에서 보관
```

설계 기준은 세 가지였다. 필요한 경로에서는 원문을 복원할 수 있어야 한다. 손상되거나 다른 일기에서 옮겨 온 암호문은 거절해야 한다. 기존 Java 21과 JPA, PostgreSQL의 저장 흐름에 통합할 수 있어야 한다. 원문 검색이나 DB 문자열 연산 요구는 없어, 암호문으로 저장하면서 해당 연산을 유지할 필요는 없었다.

보호 대상은 저장된 원문과 요청 지문이다. 제목과 생성 이미지 같은 파생 콘텐츠를 통해 내용을 추론하는 문제까지 해결하는 것은 아니다. 애플리케이션과 키가 함께 탈취된 경우, 접근 권한 검사에 오류가 있는 경우도 별도로 대응해야 한다. 이 범위를 먼저 정해야 암호화에 기대할 수 있는 효과를 판단할 수 있다.

2. 원문 복원과 변조 검증을 기준으로 방식을 골랐다

원문은 애플리케이션에서 암호화한다

Base64는 바이트를 문자열로 표현하는 Encoding(인코딩) 규칙이다. 비밀 키 없이 되돌릴 수 있으므로, 원문에 Base64만 적용해 저장해도 DB를 읽은 사람은 원문을 복원할 수 있다. 암호화한 바이트를 TEXT 열에 저장할 때는 유용하지만, 원문을 숨기는 수단은 아니다. RFC 4648의 보안 고려사항

디스크 암호화는 저장 매체를 보호한다. 그러나 정상 SQL 조회에서 원문을 반환한다면 그 결과나 논리 덤프에 담긴 원문은 그대로 남는다. PostgreSQL의 pgcrypto로 SQL에서 암복호화할 수도 있지만, 이 경우 키와 평문이 DB 서버의 처리 경계를 통과한다. 이번에는 키를 DB 밖에 두려 했으므로 **애플리케이션이 암호화한 결과만 DB에 전달하는 방식**을 선택했다. PostgreSQL 보안 제한 문서

암호화와 변경 검사를 함께 처리한다

AES는 같은 비밀 키로 암호화와 복호화를 수행하는 대칭키 알고리즘이다. AES-256의 256은 키 길이로, 32바이트를 뜻한다. AES를 어떤 방식으로 사용할지는 별도로 정해야 한다.

비교한 방식 중 AES-CBC는 원문을 암호문으로 바꿀 수 있지만, 암호문이 바뀌었는지 검증하는 기능은 없다. 이를 보완하려면 HMAC처럼 **비밀 키로 검증값을 만드는 방법**을 함께 사용해야 한다. 올바르게 구성한 CBC와 HMAC의 조합은 요구를 충족할 수 있다. 다만 앱이 아래 순서와 대상을 직접 관리해야 한다.

> CBC와 HMAC을 조합한다면
> 저장: 암호화 -> 별도 키로 암호문과 부가정보의 HMAC 계산 -> 함께 저장
> 조회: HMAC 검증 -> 성공한 경우에만 복호화

검증해야 할 정보를 빠뜨리거나, 검증 전에 복호화한 값을 사용하면 문제가 생긴다. 키도 용도에 맞게 나누어야 한다. 기존 CBC 데이터와 호환할 필요가 없었으므로, 이 조합을 직접 구성하는 대신 Authenticated Encryption with Associated Data(AEAD, 추가 데이터 인증을 지원하는 인증 암호화) 방식을 검토했다. 원문을 숨기는 연산과 데이터가 바뀌었는지 확인하는 연산을 함께 제공하는 방식이다.

AES-GCM과 ChaCha20-Poly1305는 모두 이 요구를 충족한다. Galois/Counter Mode(GCM)는 AES로 암호화하면서 변경 여부도 검증하는 운용 방식이다. 여기서 인증은 로그인 확인이 아니라 **암호문과 추가 정보가 암호화할 때의 조건에 맞는지 검사한다는 뜻**이다.

두 방식 모두 Java 21에서 사용할 수 있다. 이번에는 AES-GCM으로 저장 형식 v1을 고정했다. 선택의 근거는 인증 기능을 직접 조합할 항목을 줄이고 기존 Cipher API로 구현한다는 것이다. 이 근거는 CBC와 HMAC의 조합 대신 AEAD를 택한 이유이며, ChaCha20-Poly1305보다 AES-GCM이 우수하다는 증명은 아니다. 실제 서버의 성능 비교 결과도 선택 근거에 넣지 않았다. Java Cipher 문서

AES-128도 가능한 선택이다. AES-256은 32바이트 키 규격과 키 탐색에 대한 추가 여유를 택한 것이다. 키 길이를 늘려도 키 유출이나 nonce 재사용 같은 잘못된 사용법이 해결되지는 않는다. nonce는 다음 절에서 설명한다.

3. 원문과 함께 nonce와 인증 태그를 저장한다

원문을 암호화하기 전에 빈 값과 길이를 검사하고 앞뒤 공백을 제거한다. 그 결과를 UTF-8 바이트로 바꾼 값이 코드의 plaintextBytes다. 잘못된 유니코드 입력은 다른 문자로 대체하지 않고 실패시킨다.

같은 키에서 다시 사용하면 안 되는 nonce

Nonce(논스)는 각 암호화에 사용하는 값이다. 구현에서는 SecureRandom으로 매번 12바이트를 생성한다. 키는 여러 일기에 재사용하지만, **같은 키로 암호화할 때 nonce를 재사용하면 안 된다.**

예를 들어 같은 일기를 두 번 암호화하더라도 두 호출은 각각 새 nonce를 만든다. nonce가 달라지면 암호화 결과도 달라진다. 반대로 같은 키와 nonce를 반복 사용하면 GCM의 보안 전제가 깨져 원문 보호와 위조 방지에 문제가 생길 수 있다. nonce는 비밀 키와 역할이 다르므로, 복호화에 다시 쓸 수 있게 암호문과 함께 저장한다. NIST SP 800-38D

보안 난수를 쓴다고 중복 가능성이 사라지는 것은 아니다. 키별 암호화 사용량과 교체 정책을 관리해야 한다는 조건이 남는다. 아래는 실제 코드에서 예외 처리와 임시 배열 정리를 생략한 발췌다.

```
byte[] nonce = new byte[12];
secureRandom.nextBytes(nonce);
Cipher cipher = newCipher(
        Cipher.ENCRYPT_MODE, activeKeyId, nonce, diaryId, userId);
byte[] ciphertext = cipher.doFinal(plaintextBytes);
byte[] payload = ByteBuffer.allocate(nonce.length + ciphertext.length)
        .put(nonce).put(ciphertext).array();
return activeKeyId + ":" + Base64.getEncoder().encodeToString(payload);
```

newCipher()는 키와 nonce를 설정하고, 다음 절에서 설명할 일기 식별 정보도 인증 대상으로 넣는다. doFinal()이 실제 암호화를 수행한다.

암호문 뒤에는 이미 태그가 붙어 있다

Authentication Tag(인증 태그)는 복호화할 때 데이터가 암호화 당시의 조건에 맞는지 검사할 검증값이다. Java의 GCM 암호화에서는 doFinal() 반환값에 **암호문과 태그가 함께 들어 있다.** 위 코드의 ciphertext도 이 둘을 포함하므로 태그를 다시 붙이지 않는다. 태그 길이는 128비트, 즉 16바이트로 설정했다. AES 키의 256비트와는 다른 설정이다. Java Cipher 문서

```
저장 문자열 = keyId:Base64(payload)
payload    = nonce 12바이트 || 암호문 || 태그 16바이트
||         = 바이트를 순서대로 이어 붙인다는 표기
```

키 ID는 서버의 키 목록에서 사용할 키를 찾는 이름이다. 실제 키는 DB에 넣지 않는다. 별도 열 source_text_version에 기록하는 버전 1은 이 저장 형식을 해석하는 규칙이다.

원문 오늘은 UTF-8로 6바이트다. payload는 12 + 6 + 16 = 34바이트, Base64로 48문자다. 키 ID가 k1이면 k1:까지 총 51문자다. 입력 상한은 300 Unicode code point로, UTF-8에서 최대 1,200바이트다. 따라서 payload 상한은 1,228바이트다. code point는 유니코드 문자를 세는 단위다. 일부 이모지는 화면에 한 글자로 보여도 여러 code point를 사용한다.

4. 읽을 권한과 암호문의 유효성을 따로 확인한다

본인 상세 조회에서는 DB에서 일기를 가져온 뒤 요청자와 소유자가 같은지 먼저 확인한다. 그다음 복호화한다. **소유권 검사는 누가 읽을 수 있는지 판단하고, 암호문 인증은 저장된 값을 신뢰해도 되는지 판단한다.** 한쪽이 다른 쪽을 대신하지 않는다.

복호화의 첫 단계는 저장 형식 검사다. 지원하는 버전인지, 키 ID가 서버에 등록돼 있는지, Base64와 payload 길이가 규칙에 맞는지 확인한다. Base64를 다시 인코딩한 값이 원래 표현과 같은지도 검사한다. 이 단계는 읽을 수 있는 형식인지 확인하는 것이며, 태그 검증은 아직 수행하지 않은 상태다.

다른 일기의 암호문을 복사하면 왜 실패할까

A 일기의 저장 문자열을 B 일기의 원문 열에 복사했다고 가정하자. nonce와 암호문, 태그가 함께 옮겨지므로 내용만 검증해서는 어느 일기에 속하는지 알 수 없다. 이를 구분하려고 Additional Authenticated Data(AAD, 추가 인증 데이터)에 일기와 작성자의 식별 정보를 넣었다. AAD는 숨기지 않지만 암호문과 함께 검증하는 정보다.

```
AAD = harudle.diary.source-text:v1:<keyId>:<diaryId>:<userId>

A 일기에서 암호화: A의 일기 ID와 소유자 ID 사용
B 일기에서 복호화: B의 일기 ID와 소유자 ID 사용
결과: AAD가 달라져 태그 검증 실패
```

AAD는 암호문 안에서 꺼내 신뢰하는 값이 아니다. 조회한 일기 ID와 소유자 ID로 서버가 다시 구성한다. 따라서 암호문만 다른 일기에 복사하면 그 일기의 정보와 맞지 않아 거절된다. DB 변경 즉시 경보가 울리는 것은 아니며, 그 값을 복호화할 때 확인한다. 같은 행의 과거 암호문 재사용이나 AAD에 넣지 않은 날짜의 변경은 이 검사의 대상이 아니다.

태그가 맞아도 일기로 사용할 수 없는 값이 있다

다음은 복호화 핵심 코드다. payload 앞 12바이트를 nonce로 복사하고, 나머지 암호문과 태그를 doFinal()에 넘긴다.

```
byte[] nonce = Arrays.copyOfRange(payload, 0, 12);
Cipher cipher = newCipher(
        Cipher.DECRYPT_MODE, envelope.keyId(), nonce, diaryId, userId);
plaintextBytes = cipher.doFinal(payload, 12, payload.length - 12);
return Diary.normalizeSourceText(decodeStrictly(plaintextBytes));
```

세 번째 인자는 종료 위치가 아니라 처리할 바이트 수다. payload가 34바이트라면 위치 12부터 22바이트를 처리한다. 태그 검증에 성공하면 태그를 제외한 원문 6바이트를 반환한다.

그런데 태그는 데이터가 우리 서비스의 일기 규칙에 맞는지 판단하지 않는다. 테스트 보조 함수에서 정상 키로 공백 문자열을 직접 암호화하면 태그는 유효하지만, 공백뿐인 일기는 허용할 수 없다. UTF-8로 해석할 수 없는 바이트나 301 code point의 원문도 같은 이유로 구분해 검사해야 한다.

실패는 민감값과 원래 예외를 포함하지 않는 고정 예외로 전달한다. finally에서 임시 원문 바이트를 0으로 덮지만 원래 String과 반환 문자열, JVM 내부 복사본까지 모두 지우는 것은 아니다.

5. 짧은 원문은 SHA 지문으로도 확인할 수 있다

암호문 저장을 바꾸고 나면 요청 지문이 남는다. Idempotency(멱등성)는 같은 요청을 반복해도 의도한 효과가 중복되지 않게 하는 성질이다. 하루들은 클라이언트의 멱등성 키로 기존 요청을 찾는다. 같은 키로 다른 일기를 보내는 경우는 별도로 거절해야 하므로, 내용이 같은지 확인할 요청 지문도 저장한다.

해시를 복원하지 않고도 후보는 대조할 수 있다

기존 지문은 사용자 ID와 날짜, 원문 등을 연결한 문자열의 SHA-256이었다. 같은 입력은 같은 32바이트 해시값이 되며, 16진수로 표현하면 64자리다. 이 중간값을 **요청 SHA**라고 부른다.

```
SHA 계산 전 문자열 = v1\n<userId>\n<diaryDate>\n<normalizedDiaryText>
\n은 실제 줄바꿈을 뜻한다.
```

해시에서 임의의 원문을 복원하는 일은 어렵다. 그러나 DB에서 읽은 사용자 ID와 날짜에 원문 후보 오늘도 피곤했다를 넣어 같은 SHA를 계산하는 것은 가능하다. DB의 지문과 비교하면 후보가 맞는지 대조할 수 있다. 짧고 흔한 문장일수록 시도할 여지가 크다.

해시 입력에 더하는 공개 값인 salt도 함께 읽혔다면 같은 계산이 가능하다. **키가 없는 공격자는 대조할 값을 만들지 못하도록** 저장 지문을 바꿔야 했다.

두 번의 HMAC 중 첫 결과는 키, 두 번째 결과는 지문이다

Hash-based Message Authentication Code(HMAC, 비밀 키와 해시로 계산하는 메시지 인증 코드)는 키와 메시지로 검증값을 만든다. 후보의 요청 SHA를 알아도, 지문용 키가 없으면 DB에 저장된 HMAC과 대조할 값을 계산할 수 없다. HMAC 규격 RFC 2104

구현은 설정된 AES 키를 루트 키로 삼아 아래 두 계산을 수행한다.

```
HMAC(키=루트 AES 키, 메시지=고정 목적 문자열) = 지문용 키
HMAC(키=지문용 키, 메시지=요청 문맥 || 요청 SHA) = 저장할 HMAC 지문
```

첫 계산의 목적 문자열은 harudle.diary.request-fingerprint.key:v1이다. 원시 AES 키를 지문에 그대로 쓰는 대신 지문용 키를 계산한다. 같은 루트 키와 목적 문자열에서는 같은 32바이트 키가 나온다. 새 난수를 만드는 단계는 아니다.

둘째 계산은 아래 문맥의 UTF-8 바이트 뒤에 요청 SHA의 소문자 16진수 64자리 문자열을 ASCII 64바이트로 이어 넣는다. 마지막 콜론도 포함한다.

```
harudle.diary.request-fingerprint:v1:<keyId>:<diaryId>:<userId>:
```

최종 HMAC 32바이트를 소문자 16진수 64자리로 저장하고, 중간 요청 SHA는 따로 저장하지 않는다. SHA를 먼저 거치는 것이 HMAC의 필수 조건은 아니다. 기존 생성기가 요청 SHA를 만들고, 암호화 구현이 그 출력을 보호하는 구조다.

루트 키가 유출되면 지문용 키도 다시 계산할 수 있다. **키 파생은 루트 키 유출에 대한 독립적인 방어가 아니다.** DB만 유출되고 키와 HMAC 계산 수단은 유출되지 않은 경우의 보호다.

6. 같은 요청은 같은 지문으로 다시 계산한다

원문 암호화와 지문 계산은 같은 데이터에 서로 다른 성질을 요구한다. 암호화에서는 매번 새 nonce를 사용하므로 같은 원문도 저장 문자열이 달라진다. 이 문자열끼리 비교해서 같은 요청인지 판단할 수는 없다. 반면 HMAC 지문은 **같은 일기와 사용자, 키 ID와 키 값, 요청 내용이면 같은 값이 나와야 한다.** 지문 입력에 nonce나 암호문 바이트를 넣지 않은 이유다.

새 요청과 재요청이 만나는 지점

새 요청은 원문을 암호화한 일기를 저장하고, 요청 SHA를 HMAC으로 보호해 생성 이력에 저장한다. 관련 변경을 모두 반영하거나 모두 취소하는 처리 단위인 트랜잭션으로 묶어, 일기와 생성 이력 중 한쪽만 저장되는 상태를 피한다.

같은 멱등성 키로 재요청하면 기존 생성 이력과 일기를 가져와 소유권을 확인한다. 그다음 들어온 요청으로 HMAC 지문을 다시 계산해 저장값과 비교한다. 원문이 바뀌었다면 요청 SHA와 HMAC도 달라지므로 기존 요청의 재시도로 처리하지 않는다.

```
처음 요청
  -> 원문 암호화 -> 일기 저장
  -> 요청 SHA 계산 -> HMAC 보호 -> 생성 이력 저장

같은 멱등성 키로 재요청
  -> 기존 이력과 일기 조회 -> 소유권 확인
  -> 요청 SHA와 HMAC 재계산 -> 저장 지문과 비교
  -> 같으면 기존 상태 처리, 다르면 충돌로 거절
```

이 흐름은 DiaryCreationClaimService에 구현했다. HMAC을 계산하는 함수 자체는 저장 문자열의 형식과 키 ID를 확인할 뿐, 원문을 복호화하거나 GCM 태그를 검증하지 않는다. 그러나 전체 재요청 흐름에서는 지문 비교 이후 toClaim()에서 실제 원문을 복호화한다. 지문 계산과 재요청 전체의 처리 범위를 구분해야 한다.

AI를 호출하기 전에도 실행 조건을 확인한다

생성 실행기 ClaimedDiaryGenerationService는 AI 호출 직전에도 보호된 지문을 다시 계산한다. 저장된 생성 이력과 아래 네 조건이 맞는지 확인한다.

- 실행하려는 일기 ID가 같다.

- 요청을 가리키는 멱등성 키가 같다.

- 다시 계산한 HMAC 지문이 같다.

- 생성 상태가 진행 중을 뜻하는 PROCESSING이다.

이 검사는 다른 요청의 실행 정보를 섞거나, 저장된 조건과 맞지 않는 요청을 실행하는 것을 거절한다. 동시 요청에서 외부 호출이 반드시 한 번만 일어나게 하는 전체 설계를 대신하지는 않는다. 잠금과 트랜잭션, 상태 변경도 별도로 맞아야 한다.

이 비교는 키를 교체한 뒤에도 유지돼야 한다. 이를 위해 **암호문에 기록한 키 ID로 저장 당시의 키**를 찾는다.

7. 키를 바꿔도 기존 일기를 읽을 수 있어야 한다

설정에는 여러 키를 보관할 수 있다. 이 중 활성 키는 새 암호화에 사용할 키다. 키 목록에 k1과 k2를 함께 둔 채 활성 키를 k2로 바꾸면, 새 일기는 k2로 암호화하고 기존 k1: 데이터는 k1으로 읽는다. 재시도 지문도 저장된 키 ID를 사용하므로 기존 값을 재현할 수 있다.

```
키 목록: k1, k2 / 활성 키: k2
기존 k1: 데이터 -> k1으로 복호화하고 HMAC 지문 재계산
신규 작성       -> k2로 암호화하고 HMAC 지문 생성
```

활성 키 변경은 기존 행의 재암호화가 아니다. 또한 앱은 메모리에 키 목록을 보관하므로 호스트 설정 파일만 바꿔도 실행 중인 앱이 갱신되는 것은 아니다. 현재 운영 절차는 쓰기를 멈추고 처리 중인 요청이 끝난 뒤, 키 등록과 검증, 재배포를 거쳐 새 키를 사용하도록 한다.

길이가 맞는 키와 예전 데이터를 읽을 수 있는 키는 다르다

키 설정에서는 ID 문법과 Base64, 32바이트 길이를 검사한다. 하지만 같은 ID에 엉뚱한 32바이트 키를 넣어도 이 검사는 통과할 수 있다. 그래서 이전에 만든 알려진 문장의 암호문과 HMAC 검증 자료로 실제 키가 맞는지 확인한다. 잘못된 키로 검증 자료까지 새로 만들면 예전 데이터 복원 가능성을 확인할 수 없으므로, 기존 자료를 보존해야 한다.

키를 앱에서 직접 보관하면 이런 검증과 복구 절차도 직접 관리해야 한다. Key Management Service(KMS, 키 관리 서비스)를 사용하면 접근 제어와 감사, 키 보관을 개선할 수 있지만 권한 설정과 네트워크, 장애 대응을 운영에 추가한다. KMS와 AES-GCM은 함께 사용할 수도 있다. 이번에는 추가 외부 호출을 필수로 만들지 않는 환경별 키 목록으로 시작하고, 그에 따른 보관과 교체 책임을 문서화했다.

별도 무작위 HMAC 키를 두면 권한을 더 세밀하게 나눌 수 있지만, 두 키 목록의 보존과 교체 관계를 관리해야 한다. 현재는 같은 앱에서 암호화와 지문 계산을 수행하므로 목적을 나누어 키를 파생하는 방식을 선택했다.

기존 평문과 백업은 별도로 전환한다

신규 저장을 암호화했다고 기존 평문이 사라지지는 않는다. 최초 구현은 새 원문과 지문을 보호하면서 기존 평문과 SHA를 읽는 호환 경로를 남겼다. 후속 작업에서는 원문과 저장 버전, 지문을 함께 바꾸는 별도 배치와 평문 호환 경로 제거를 다뤘다.

이미 암호화한 일기를 향후 다른 키로 재암호화한다면 HMAC 지문도 함께 바꿔야 한다. 둘이 다른 키를 기준으로 남으면 정상 재시도가 충돌할 수 있다. 원문과 버전, 지문을 같은 트랜잭션으로 전환하고 진행 중인 생성과 전환이 겹치지 않게 해야 한다.

구키를 없애기 전에는 현재 DB뿐 아니라 보존 중인 백업의 복원에 필요한지도 확인해야 한다. 반대로 예전 덤프에 평문과 SHA가 남아 있다면 새 암호화 키를 쓰거나 구키를 폐기해도 그 위험은 없어지지 않는다. 전환 코드를 구현한 사실과 실제 전환, 복원, 폐기 작업의 완료 여부를 따로 기록해야 하는 이유다.

8. 성공한 왕복보다 실패해야 할 조건을 검증한다

암호화 후 복호화한 결과가 같다는 왕복 테스트는 필요하지만, 양쪽에 같은 실수가 있으면 통과할 수 있다. 테스트는 앞에서 정한 보호 목표에 맞춰 **거절해야 할 입력과 보존해야 할 기능**을 함께 확인하도록 구성했다.

보호 목표를 테스트 조건으로 바꾸기

- **손상된 값을 거절한다.** nonce와 암호문, 태그의 한 비트를 각각 바꾼 경우, 다른 일기 ID나 사용자 ID로 읽는 경우에 실패하는지 확인한다. 정상 태그를 가진 잘못된 UTF-8, 공백 원문, 길이 초과 원문도 거절해야 한다.

- **권한을 먼저 확인한다.** 타인의 일기가 손상돼 있어도 복호화 오류보다 소유권 오류가 먼저 나야 한다. 읽을 권한이 없는 데이터는 복호화하지 않는 순서를 확인한다.

- **동일 요청을 재현한다.** 같은 입력의 HMAC이 같은지, 일기나 사용자가 달라지면 다른지, 활성 키 변경 뒤에도 기존 지문을 재현하는지 확인한다. 다른 구현으로 미리 계산한 값과도 비교한다.

- **저장과 실패 처리를 확인한다.** DB 통합 테스트로 실제 원문 열에 암호문이, 지문 열에 요청 SHA 대신 HMAC이 들어가는지 검사한다. 재요청 처리 중 복호화가 실패하면 상태 변경도 함께 롤백되는지 확인한다.

로그와 응답은 서로 다른 출력 경로다

요청과 응답 객체의 toString()에서는 원문을 가렸다. 그러나 log.info("{}", request.sourceText())처럼 필드를 직접 출력하면 이 처리를 거치지 않는다. JSON으로 바꾸는 경로도 toString()과 다르다.

따라서 실제 로거와 인코더를 사용하는 테스트로 가림 처리 범위를 확인했다. 동시에 작성자 응답과 AI 요청에는 필요한 원문이 유지되는지도 검사했다. 예외 메시지에는 원문과 키, 암호문을 넣지 않는다. 저장 열을 암호화한 뒤에도 원문이 어디로 전달되는지 따라가야 출력 경로의 누출을 찾을 수 있다.

다른 서비스에 적용한다면 먼저 복원이 필요한 경로와 키 보관 위치를 정한다. 다음으로 원문에서 만든 지문과 로그를 확인한다. 저장 형식과 키 교체 절차를 정한 뒤 변조와 권한 오류, 실패 롤백을 검증한다. 마지막으로 기존 데이터와 백업의 전환 대상을 확인한다. 이 순서가 암호화 함수를 기존 기능과 운영 절차에 연결하는 출발점이 된다.

> 성능 비교 벤치마크는 수행하지 않았다. 도입 시에는 입력 크기와 처리량, 실제 서버 환경을 기준으로 측정해야 한다.

구현과 참고 자료

아래 자료의 확인일은 2026-11-09다.

신규 저장 암호화는 PR #442에서, 기존 데이터 전환은 후속 PR #450에서 다뤘다. 코드 설명은 후속 작업의 커밋 7f54ca12를 기준으로 한다. 운영 데이터 전환을 완료했다는 의미는 아니다.

- 설계 결정 ADR: 대안 비교와 보호 범위, 키 관리 결정.

- 암호화와 HMAC 구현, 암호화 테스트, HMAC 지문 테스트.

- 규격과 API 동작: NIST SP 800-38D, Java 21 Cipher, RFC 2104, RFC 4648.